

Reviewer Pack — Minimal Rank of Sheaves over Function Fields vs DPLL Refutat...

Entry 189ea0e5c097 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 13:21:57 UTC

Minimal Rank of Sheaves over Function Fields vs DPLL Refutation Tree Width

Entry ID: 189ea0e5c097 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 13:21:57 UTC

1. Conjecture

Field A (mathematical branch): Sheaf Theory **Field B** (complexity object): Complexity Theory: DPLL Refutation Complexity

Statement:

[‘For every satisfiable CNF formula F with m clauses on n variables, define the sheaf rank of F as the minimal rank of a sheaf associated to the zero set of its clause-indicator polynomial. Then, for all instances of F , the width of the DPLL refutation tree is upper-bounded by a function of the sheaf rank, specifically, $\Theta(\log(n^m + m))$.’, ‘Equivalently, the minimal sheaf rank of the clause-indicator polynomial for a CNF formula implies an upper bound on the width of its DPLL refutation tree.’, ‘Additionally, this relationship holds for all instances with sheaf rank less than or equal to some constant C .’]

Rationale (proposer’s reasoning):

[‘Sheaf theory provides a powerful tool for studying geometric and algebraic properties in complex spaces. Applying it to CNF formulas allows for the exploration of new structural features that could be exploited in complexity analysis.’, ‘Previous research has shown that certain geometric properties can give insights into complexity measures like circuit depth or refutation tree width. This conjecture proposes a connection between sheaf theory and DPLL refutation trees, which are fundamental to propositional logic.’, ‘If true, this would provide a new approach to analyzing the complexity of propositional satisfiability and could potentially lead to improved algorithms.’]

Taxonomy category: Sheaf_Theory_vs_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8b6901dd9642a137

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the average DPLL refutation tree width is within $\Theta(\log(n^m + m))$ of the sheaf rank for all tested CNF formulas, and if no seed results in an average width greater than $2 * \Theta(\log(n^m + m))$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "sheaf theory" AND "DPLL refutation tree width" - "minimal rank of sheaves" AND CNF formula" - "zero set of clause-indicator polynomial" AND upper bound on DPLL refutation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find max pivot in column i
        max_idx = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_idx][i]):
                max_idx = j

        # Swap rows i and max_idx
        A[i], A[max_idx] = A[max_idx], A[i]
        b[i], b[max_idx] = b[max_idx], b[i]

        # Eliminate entries below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]

    # Back-substitution
```

```

x = [0] * n
for i in range(n-1, -1, -1):
    x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]

return x

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    if len(A) == 1:
        return A[0][0]
    det = Fraction(0)
    sign = 1
    for i in range(len(A)):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def sheaf_rank(n, m):
    # Construct the clause-indicator polynomial
    x = [Fraction(1) if i % 2 == 0 else Fraction(-1) for i in range(n)]
    term = Fraction(1)
    for var in range(1, n+1):
        term *= (1 + x[var-1])

    # Compute the determinant of the matrix associated with the polynomial
    A = [[term.coeff(x[i], j) for j in range(m)] for i in range(n)]
    det = determinant(A)

    return abs(det).numerator

def construct_dpll_tree(n, m):
    clauses = [random.sample(range(1, n+1), 2) for _ in range(m)]
    tree = []
    stack = [(clauses, [])]
    while stack:
        remaining_clauses, assignment = stack.pop()
        if not remaining_clauses:
            tree.append(assignment)
            continue
        var = random.choice(range(1, n+1))
        true_assignments = [a for a in assignment if var not in a]
        false_assignments = [a + [(var, True)] for a in assignment] + [a + [(var, False)] for a in assignment]
        stack.append((remaining_clauses, true_assignments))
        stack.append((remaining_clauses, false_assignments))
    return tree

```

```

def dpll_tree_width(tree):
    max_width = 0
    for path in tree:
        width = len(path)
        if width > max_width:
            max_width = width
    return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = random.randint(1, n * (n - 1))
        rank = sheaf_rank(n, m)
        tree = construct_dpll_tree(n, m)
        width = dpll_tree_width(tree)

        results.append({
            "n": n,
            "m": m,
            "rank": rank,
            "width": width
        })

    mean_rank = sum(result["rank"] for result in results) / len(results)
    mean_width = sum(result["width"] for result in results) / len(results)

    if all(width <= 2 * math.log(n*m + m, 2) for n, m, _, width in results):
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "DPLL tree width exceeds expected bound"

    return {
        "metric_name": "dpll_tree_width",
        "metric_value": mean_width,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_width = sum(result["metric_value"] for result in results) / len(results)

```

```

support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
    first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"DPLL tree width exceeds expected bound\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c992c25b.py", line 146, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c992c25b.py", line 111, in run_trial
    rank = sheaf_rank(n, m)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c992c25b.py", line 74, in sheaf_rank
    A = [[term.coeff(x[i], j) for j in range(m)] for i in range(n)]
          ^^^^^^^^^
AttributeError: 'Fraction' object has no attribute 'coeff'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying whether the average DPLL refutation tree width meets the pre-registered support condition. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14666
2	propose	ollama_remote	glm4:latest	0	0	12853
3	propose	ollama_remote	glm4:latest	0	0	11830

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	preregistration	ollama_remote	glm4:latest	0	0	5720
5	novelty	ollama_remote	glm4:latest	0	0	4645
6	novelty	ollama_remote	glm4:latest	0	0	8671
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15380
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7531
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12580
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16194
11	judge	ollama_remote	glm4:latest	0	0	8133

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118203 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/189ea0e5c097.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/189ea0e5c097.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/189ea0e5c097.tar.gz* (if generated)